

Міністерство освіти і науки України
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
“КИЄВО-МОГИЛЯНСЬКА АКАДЕМІЯ”
Кафедра мультимедійних систем
факультету інформатики

Трасування променів у воксельному просторі

Текстова частина до курсової роботи
за спеціальністю “Інженерія програмного забезпечення”

Керівник курсової роботи
старший викладач,
Борозенний С. О.

(підпис)

“ ____ ” _____ 2026 р.

Виконав студент Гнутов О. В.

“ ____ ” _____ 2026 р.

Київ 2026

Міністерство освіти і науки України

НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
“КИЄВО-МОГИЛЯНСЬКА АКАДЕМІЯ”

Кафедра мультимедійних систем
факультету інформатики

ЗАТВЕРДЖУЮ

Зав. кафедри мультимедійних систем,
доцент, кандидат наук,
_____ Жежерун О.П
(підпис)
“ ____ ” _____ 2025 р.

ІНДИВІДУАЛЬНЕ ЗАВДАННЯ

на курсову роботу

студенту Гнутову Олександру 3 курсу
факультету інформатики

ТЕМА: Трасування променів у воксельному просторі

Зміст ТЧ до курсової роботи:

Індивідуальне завдання

Вступ

1. Аналіз предметної області та постановка задачі
2. Методи та особливості реалізації трасування у воксельному просторі
3. Реалізація алгоритму трасування променів у воксельному просторі

Висновок

Список використаних джерел

Додатки

Дата видачі “ ____ ” _____ 2026 р.

Керівник _____ Завдання отримав: _____

КАЛЕНДАРНИЙ ПЛАН ВИКОНАННЯ КУРСОВОЇ РОБОТИ

№ з/п	ПЕРЕЛІК РОБІТ	Термін виконання	Дата ознайомлення наукового керівника	Підпис наукового керівника	Примітки
1.	Вибір теми, затвердження її на засіданні кафедри та закріплення наукового керівника. Узгодження календарного графіка підготовки курсової роботи. Ознайомлення студента з критеріями оцінювання курсової роботи	14 грудня 2025			
2.	Вивчення джерел літератури, матеріалів архівів, періодичних видань, збір та узагальнення фактів, даних	15 грудня 2025 — 31 січня 2026			
3.	Складання плану курсової роботи та узгодження з науковим керівником	31 січня 2026			
4.	Написання розділів роботи	1 лютого — 10 березня 2026			
5.	Проміжний контроль виконання роботи	1 лютого 2026			
6.	Написання курсової роботи в цілому, ознайомлення з її першим варіантом наукового керівника	11 лютого — 10 березня 2026			
	Розділ 1 (постановка проблеми, теоретичні основи, огляд літературних джерел)	25 лютого 2026			
	Розділ 2 (аналітично-дослідницька частина)	1 березня 2026			
	Розділ 3 (проєктно-рекомендаційна частина)	10 березня 2026			
7.	Повне завершення написання курсової роботи, оформлення її згідно з вимогами й подання на відгук науковому керівнику	17 березня 2026			
8.	Подання курсової роботи для перевірки письмових робіт студентів НаУ-КМА на відповідність вимогам академічної доброчесності	24 березня 2026			
9.	Публічний захист курсової роботи перед екзаменаційною комісією	6–8 квітня 2026			

Графік узгоджено “_____” _____ 2025 р.

Науковий керівник: Борозенний Сергій Олександрович

Виконавець курсової роботи: Гнутов Олександр Володимирович

Зміст

АНОТАЦІЯ	1
ВСТУП	2
1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ	4
1.1 Поняття вокселя та воксельних структур даних	4
1.2 Методи трасування променів у дискретному просторі	5
1.3 Проблеми, що вирішує воксельне представлення	7
1.4 Постановка задачі	7
2 МЕТОДИ ТА ОСОБЛИВОСТІ РЕАЛІЗАЦІЇ ТРАСУВАННЯ У ВОКСЕЛЬНОМУ ПРОСТОРІ	8
2.1 Алгоритм DDA	8
2.2 Fast Voxel Traversal Algorithm (Amanatides and Woo)	8
2.3 Структури даних для трасування на GPU	8
2.3.1 Лінійна BVH	8
2.3.2 3D текстура	8
2.3.3 3D MIP-map як аналог octree	8
2.3.4 Загальний воксельний об'єм (World Volume)	8

2.4	Оптимізації	8
3	РЕАЛІЗАЦІЯ АЛГОРИТМУ ТРАСУВАННЯ ПРОМЕНІВ У ВОКСЕЛЬНОМУ ПРОСТОРІ	9
3.1	Створення базових структур даних та матеріалів на базі дви- гуна Bevy мовою Rust	9
3.2	Реалізація повного пайплайну відкладеного рендерингу для тра- сування у воксельному просторі	9
3.3	Написання алгоритму трасування шейдерною мовою WGSL . .	9
	ВИСНОВКИ	10
	СПИСОК ЛІТЕРАТУРИ	10
	ДОДАТКИ	12

АНОТАЦІЯ

ВСТУП

Сучасні комп'ютери, і особливо їх графічні процесори, можуть обробляти все більше даних за більш короткий, що відкриває нові можливості в галузі комп'ютерної графіки. Однак, більшість відеоігор та інших графічних застосунків досі використовує більш продуктивний, але менш фізично-коректний і реалістичний метод рендерингу — растеризацію, замість більш ресурсно-затратної технології — трасування променів.

Растеризація — це процес трансформації векторних даних (зазвичай множини трикутників) в растрове зображення, яке графічний процесор по пікселю відмальовує на екрані. В той же час трасування променів (*англ. ray tracing*) — це зовсім інший підхід, який полягає в отриманні інформації про об'єкти рендерингу через кидання променю з кожного пікселя на їх поверхню. Із самого означення зрозуміло, що наївна реалізація алгоритму трасування променів досить ресурсно-затратна, і тому вимагає оптимізацій для того, щоб можна було використати цей алгоритм на практиці.

Метою цієї роботи є розглянути основні поняття, що стосуються трасування променів, в тому числі воксельного, різні методи оптимізації рей трейсингу, їх поєднання з методом трасування у воксельному просторі, а також переваги та недоліки цього методу.

Дана робота ставить на меті виконання наступних завдань:

- Аналіз та порівняння різних методів трасування у дискретному та неперервному просторі;
- Реалізація алгоритму трасування воксельних об'ємів і створення проміжних даних для подальшого рендерингу;
- Реалізація пост-процесингового алгоритму відкладеного (*англ. deferred*) трасування у воксельному просторі з використанням загального

воксельного об'єму, створеного на основі інших об'єктів 3D-сцени;

- Дослідження додаткових методів оптимізації трасування воксельної сітки;
- Створення кінцевого застосунку, що демонструє всі наведені вище алгоритми рендерингу на прикладі воксельних 3D моделей.

1 АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ПОСТАНОВКА ЗАДАЧІ

1.1 Поняття вокселя та воксельних структур даних

Воксель (*англ.* **volume**) + **pixel** — одиничний елемент *воксельного* простору, що являє собою решітку з елементів фіксованого розміру. У тривимірному просторі є аналогом пікселя у двовимірному. У процесі трасування променів він грає роль як *візуального елемента*, оскільки 3D моделі у даному проєкті будуть самі будуть складатися з фіксованої сітки вокселів, так і *логічного елемента*, так як воксель є частиною загального воксельного об'єма (World Volume), який буде використовуватися у відкладеному рендерингу. [1]

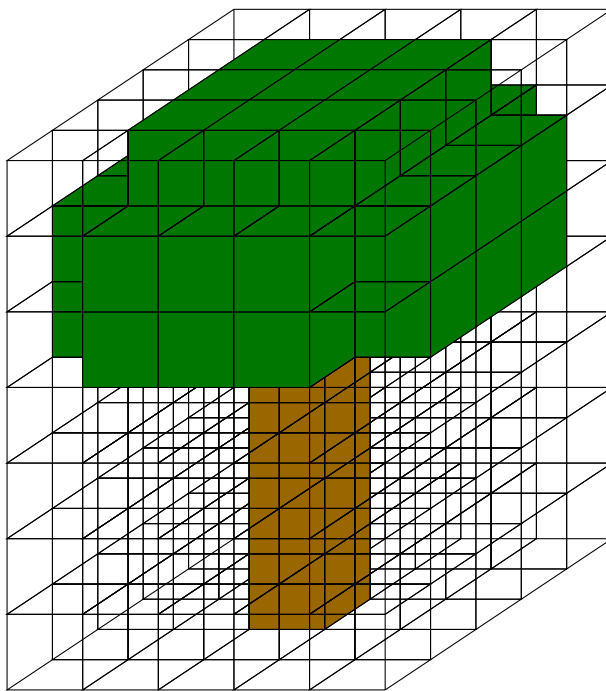


Рис. 1.1:
Воксельна модель дерева, представлена у вигляді тривимірного масиву вокселів.

Найпростішою формою представлення воксельного об'єму є однорідний **тривимірний масив**. Це масив розміру $N \times M \times P$, де N, M, P — це розміри обмежувальної коробки воксельної моделі. У реалізації в шейдерній мові він може бути представлений у вигляді лінійного масиву, індекс якого за координатами x, y, z буде рахуватися за формулою: $x + y \times N + z \times N \times M$; або ж у вигляді 3D-текстури, з якою можна працювати за допомогою вбудованої функції `textureLoad(x, y, z, L)`, де L — це MIP-рівень, який ми пізніше використаємо для подальшої оптимізації.

Однією із більш просунутих структур даних для зберігання воксельного об'єму є **октодерево** (*англ. octree*) — дерево, в якому кожен вузол має рівно вісім нащадків і представляє собою кубічний об'єм простору, який розділяється на вісім менших кубів при досягненні певного критерію розбиття; в нашому випадку — якщо цей об'єм рекурсивно містить хоча б один “твердий” воксель. Octree дозволяє ефективно зберігати та обробляти великі об'єми даних, оскільки дозволяє зберігати інформацію лише про ті частини простору, які містять важливі дані, і пропускати порожні області. [2]

Однак, для оптимізації пам'яті використання октодерева на GPU є нелегкою задачею через відсутність у шейдерних мовах вказівників, щоб не зберігати порожні області воксельного об'єму у пам'яті. Проте в цій роботі для зберігання воксельного об'єму буде використана 3D текстура з MIP-рівнями, що дозволить принаймні оптимізувати прохід по воксельному об'єму за допомогою рівнів октодерева.

1.2 Методи трасування променів у дискретному просторі

Оскільки трасування променів являє собою важке обчислення, особливо для моделей, що складаються з великої кількості полігонів, а також сцен із великою кількістю об'єктів, існує два методи для оптимізації цього алгоритму: побудова ієрархії обмежувальних об'ємів (*англ. BVH*) для об'єктів сцени та розділення простору на дискретні об'єми з вокселів. Детально обидва методи ми розглянемо пізніше. [3]

В даній роботі ми розглянемо та опишемо таку структуру даних, як загальний (світовий) воксельний об'єм (*англ.* **World Voxel Volume**), що являє собою великий ($\approx 1000^3$ вокселів) масив із вокселів, утворений вокселізацією усіх об'єктів на сцені. Він має багато переваг для наших цілей:

- просте та зручне представлення на GPU за допомогою єдиної 3D текстури;
- підтримка оптимізації октодерева (через MIP-рівні текстури);
- відсутня побудова дерева та його балансування;
- вокселізація у загальний об'єм воксельних 3D моделей (наприклад, створених у програмі MagicaVoxel), навіть *орієнтованих* (до яких застосована матриця повороту), які ми використаємо для нашого застосування, є тривіальною.

В той же час, такий спосіб має переваги і над лінійним представленням воксельних (або вокселізованих) об'єктів сцени в масиві:

- відсутня потреба у зберіганні додаткових структур даних (таких як BVH) для кожного об'єкту сцени, оскільки інакше проходження по лінійному масиву з багатьох об'єктів, не збалансованих за відстанню, буде більш ресурсозатратним;
- простота реалізації алгоритму трасування променів, оскільки всі об'єкти сцени зберігаються в єдиному просторі;
- можливість зменшення використаної пам'яті за рахунок побітового представлення вокселя в загальному об'ємі (1 — твердий воксель, 0 — порожній).

Однак даний метод має і свої недоліки:

- обмеження сцени розмірами воксельного об'єму; однак для великих процедурних світів може бути використана система чанків;
- побітове представлення вокселів хоч і зменшує використану пам'ять у рази, світовий об'єм для великої сцени може важити більше 200 MiB, при цьому втрачається інформація про колір вокселя та інші його атрибути, що в свою чергу перешкоджає реалізації більш реалістичних технік в трасуванні променів, таких як глобальне освітлення (*англ.* **Glo-**

bal Illumination, GI);

- розмір одиничного вокселя — пропорційний використуваній пам'яті GPU та швидкості, але обернено пропорційний точності обчислення.

Тому цей метод в першу чергу орієнтований на швидкодію, простоту реалізації та подібну до фізичної (а не цілком засновану на фізичних явищах) графіку.

1.3 Проблеми, що вирішує воксельне представлення

Якщо говорити про сучасні ігри та інші графічні застосунки, то вони великою мірою працюють із полігональними 3D-моделями, які в свою чергу є одними з найбільш важких для обчислення об'єктів трасування променів. Того для полігональних моделей вокселізація і розташування у загальному об'ємі є дуже ефективним методом оптимізації:

- незалежно від кількості полігонів, точність обчислення все ще буде залежати від розміру одиничного вокселя; це впливає лише на швидкість процесу вокселізації;
- для далеких від камери об'єктів можлива оптимізація MIP-рівнів, на кшталт рівнів деталізації (*англ.* **Level of Detail, LOD**) в процесі рендерингу растеризацією.

1.4 Постановка задачі

2 МЕТОДИ ТА ОСОБЛИВОСТІ РЕАЛІЗАЦІЇ ТРАСУВАННЯ У ВОКСЕЛЬНОМУ ПРОСТОРИ

2.1 Алгоритм DDA

2.2 Fast Voxel Traversal Algorithm (Amanatides and Woo)

2.3 Структури даних для трасування на GPU

2.3.1 Лінійна BVH

2.3.2 3D текстура

2.3.3 3D MIP-map як аналог octree

2.3.4 Загальний воксельний об'єм (World Volume)

2.4 Оптимізації

3 РЕАЛІЗАЦІЯ АЛГОРИТМУ ТРАСУВАННЯ ПРОМЕНІВ У ВОКСЕЛЬНОМУ ПРОСТОРІ

- 3.1 Створення базових структур даних та матеріалів на базі двигуна Bevy мовою Rust
- 3.2 Реалізація повного пайплайну відкладеного рендерингу для трасування у воксельному просторі
- 3.3 Написання алгоритму трасування шейдерною мовою WGSL

ВИСНОВКИ

СПИСОК ЛІТЕРАТУРИ

- [1] Вікіпедія — вільна енциклопедія. Воксель. — 2025. — URL: <https://uk.wikipedia.org/wiki/Воксель>.
- [2] Laine Samuli, Karras Tero. Efficient sparse voxel octrees // Proceedings of the 2010 ACM SIGGRAPH Symposium on Interactive 3D Graphics and Games. — I3D '10. — New York, NY, USA : Association for Computing Machinery, 2010. — P. 55–63. — URL: <https://doi.org/10.1145/1730804.1730814>.
- [3] Yagel R., Cohen D., Kaufman A. Discrete ray tracing // IEEE Computer Graphics and Applications. — 1992. — Vol. 12, no. 5. — P. 19–28.

ДОДАТКИ